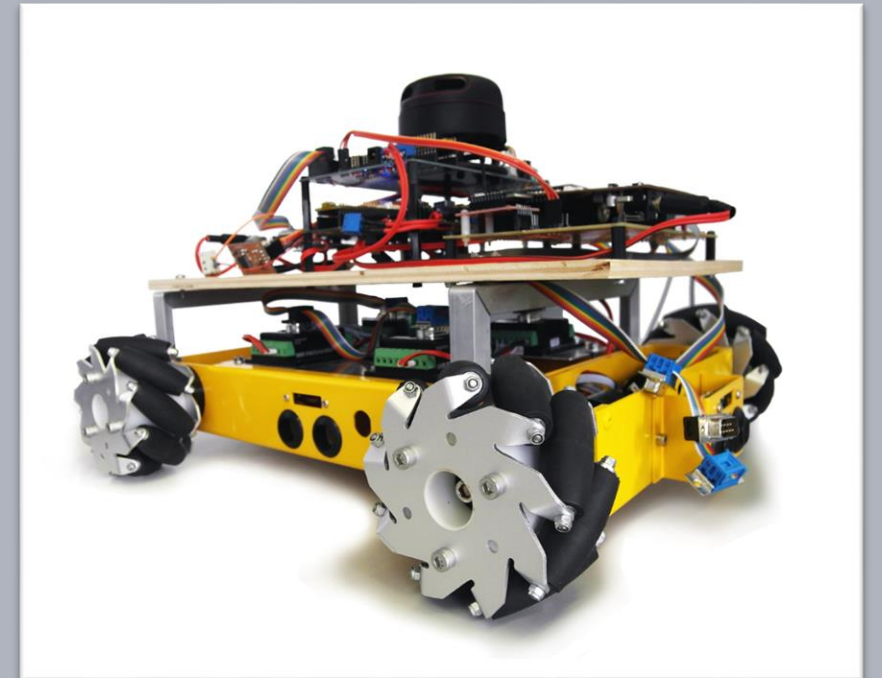


Advanced Embedded Architectures and Applications

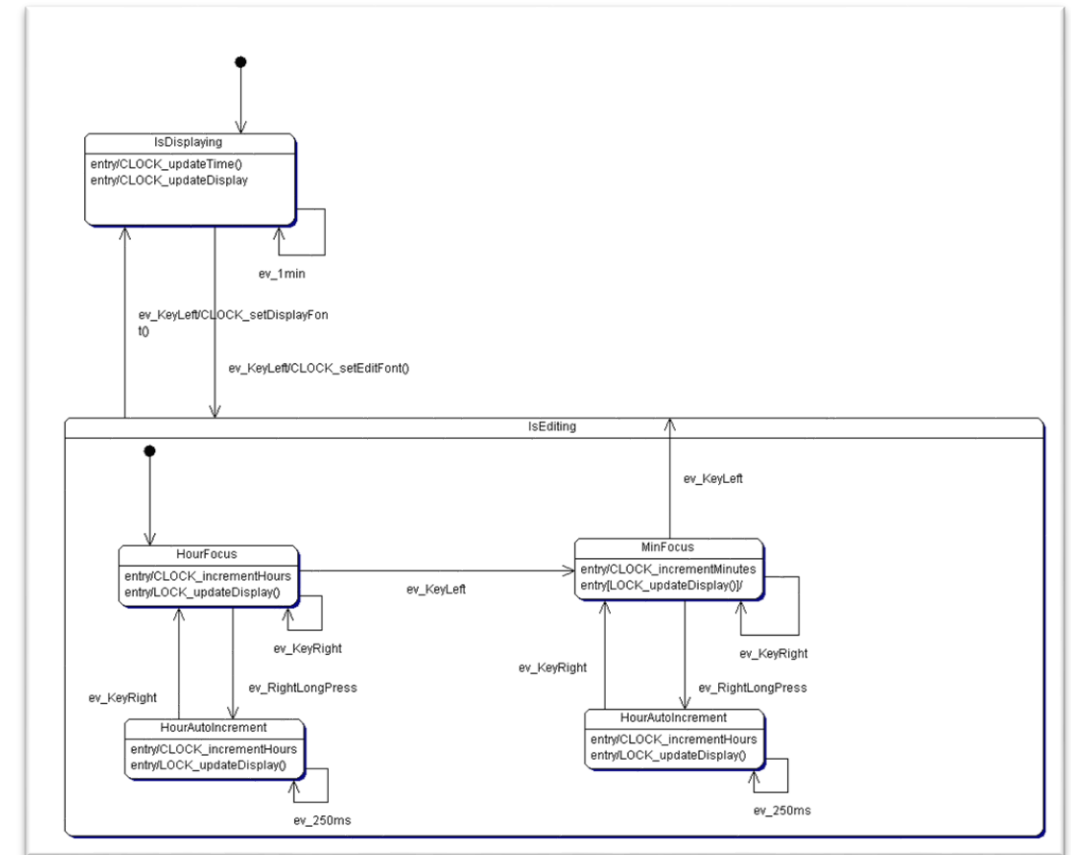
Extended State Machines

Prof. Dr.-Ing. Peter Fromm



Content

- Hierarchical State Machines
- Active Objects
- Active Objects and RTE
- Some implementation challenges
- State Machine Testing



Last Week – Switch Case Coding Pattern

```
RC_t MP3_ProcessEvent(EventMaskType nextEvent) {
    switch (MP3__player.m_state)
    {
        case MP3STATE_ISSTOPPED :
            if (nextEvent & ev_Button1)
            {
                MP3__prevSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button2)
            {
                MP3__nextSong();
                MP3__showSong();
            }
            else if (nextEvent & ev_Button3)
            {
                MP3__playSong();
                MP3__showSong();
                MP3__player.m_state = MP3STATE_ISPLAYING;
            }
            break;

        case MP3STATE_ISPLAYING :
            //...
```

Usually the states act as cases, as this makes the structure easier to understand, as compared to using the events as case.

Last Week – Transition Table Pattern

```
const STATE_stateTransition_t STATE_Alive_Transitions[] = {
/* FromState      Event      ToState      Guard      Action      */
{ state_ok,      ev_tick,      state_ok,      STATE__Alive_Timeout_OK, STATE__tick      },
{ state_ok,      ev_tick,      state_error,   STATE__Alive_Timeout_NOK, 0,      },
{ state_ok,      ev_trigger,   state_ok,      STATE__Alive_Timeout_OK, STATE__resetTimer},
{ state_ok,      ev_trigger,   state_error,   STATE__Alive_Timeout_NOK, 0      },
{ state_ok,      ev_restart,   state_ok,      0,      STATE__resetTimer},

{ state_error,   ev_tick,      state_error,   0,      STATE_tick      },
{ state_error,   ev_restart,   state_ok,      0,      STATE_resetTimer },

{ 0, 0, 0, 0, 0} /* End of table record - must be unique */
};

const STATE_stateAction_t STATE_Alive_StateActions[] = {
/* State      Entry      Exit */
{ state_ok,      STATE__PowerOn,      0      },
{ state_error,   STATE__PowerOff,      0      },

{ 0, 0, 0} /* End of table record - must be unique */

};
```

Last Week – Transition Table Pattern

```
void processEvent(event_t ev)

    state_t currentState = getState()

    while (not endOfTransitionList)
        transition = transition_table[i++];
        if (transition.fromState == currentState && transition.event == ev)

            state_t toState = transition.toState
            guard_t guard = transition.guard
            action_t action = transition.action

            if (guard == NULL || guard()==TRUE)
                while (not endOfStateActionList)
                    stateActions = action_table[j++]
                    if (stateActions.state == toState)
                        entry = stateActions.entry;
                    if (stateActions.state == currentState)
                        exit = stateActions.exit;

                if (exit != NULL) exit();
                if (action != NULL) action();
                if (entry != NULL) entry;

            setState(toState)
```

Search
transition

Search
state
actions

Execute
actions

Set state

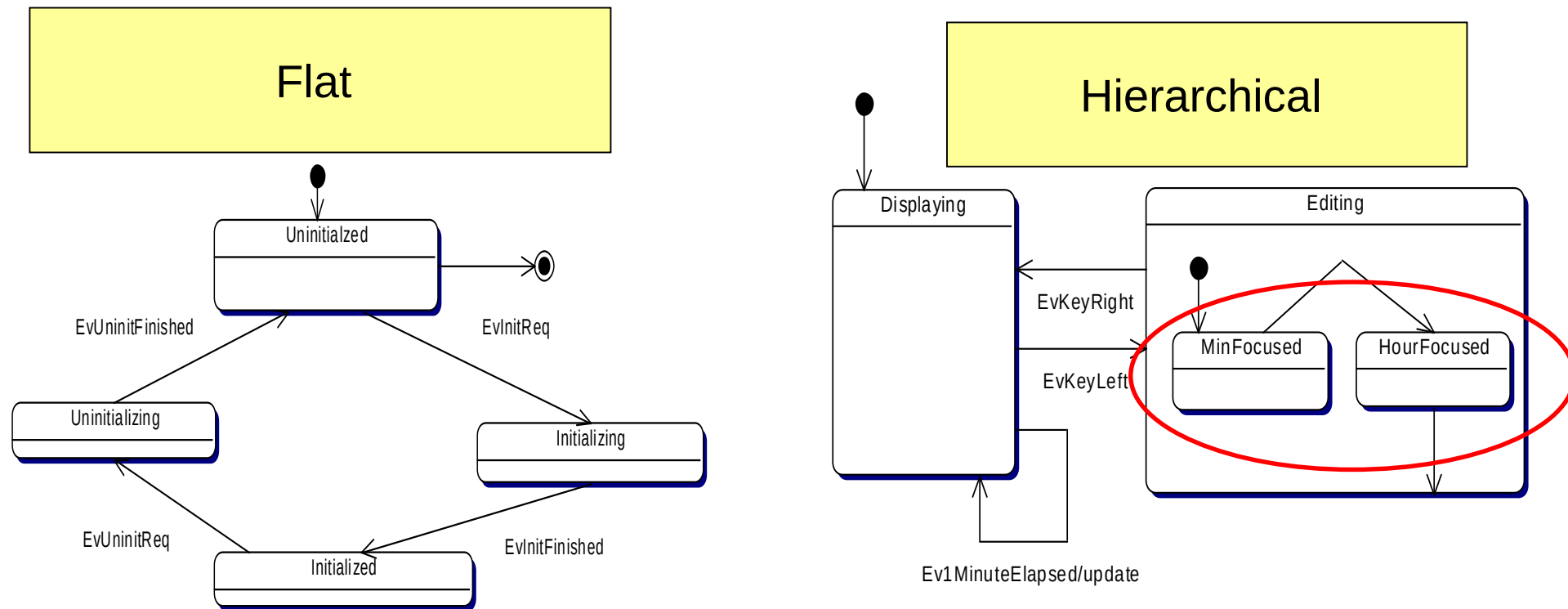
Last Week

	Switch Case	Full Transition	Explicit Transition	Two Layer
Performance	++	--	++	+
Flexibility	++	++	- (3)	++
Maintainability	--	++	+/-	++
Memory Footprint	++	+/-	+/-	+
Implementation	+ (1)	- (2)	+/-	- (2)

- (1) Good for very simple state machines
- (2) State machine code is complex, but can be used for different implementations.
Maintaining the table is simple.
- (3) For many statemachines, a single transition per state/event is sufficient

Complexity of State Machines

UML State Machine implementation is heavily depending on the complexity of the UML model (flat vs. hierarchical, pseudo states, entry/exit/do, internal transitions)



Complexity of State Machines

Code Generation Pattern

UML modeling of state charts	State Nesting	entry, exit, do, internal, history	join/fork, concurrent states, diagram nesting
Minimal:	flat		
Medium:	flat		
Full UML:	hierarchical		

Coding Pattern

A pattern being able to represent hierarchical state machines becomes very complex

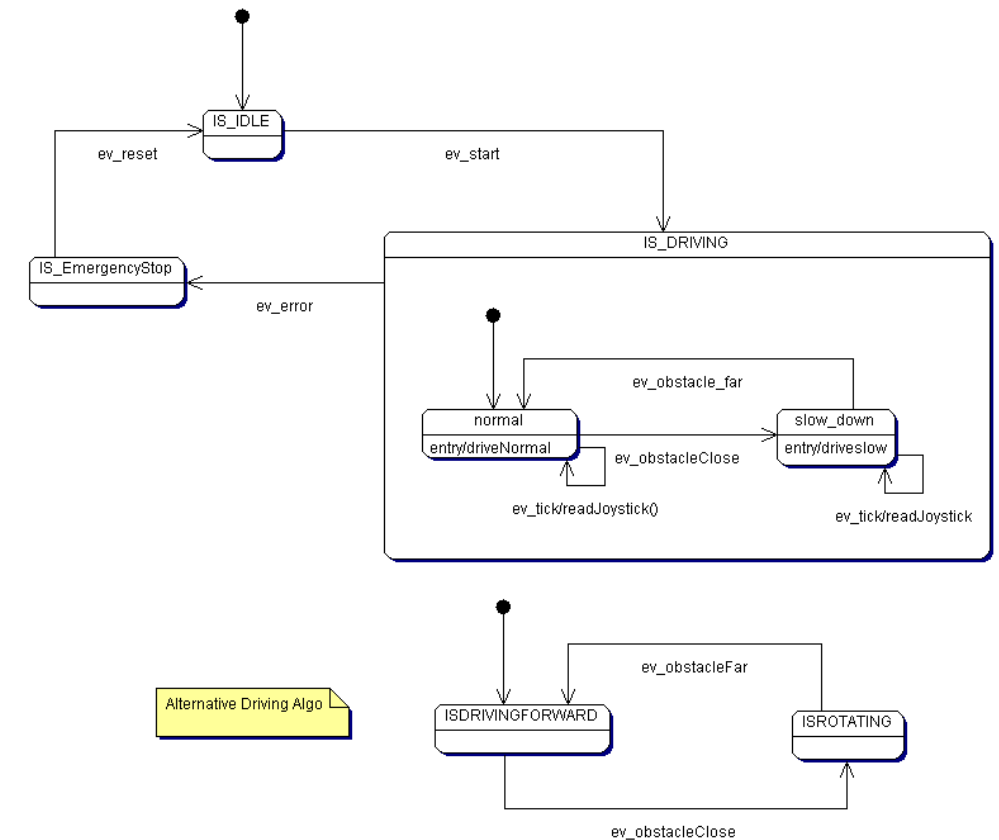
- History of states needs to be considered
- State changes may occur on every level
- A state change on a higher level usually has a consequence for the lower levels
- The values of several state variables have to be synchronized
- ...

As a consequence:

- ➔ Always try to implement flat state machines (if possible).
- ➔ Redesign hierarchical state machines to become flat state machines (if possible)

First example – Car Control System

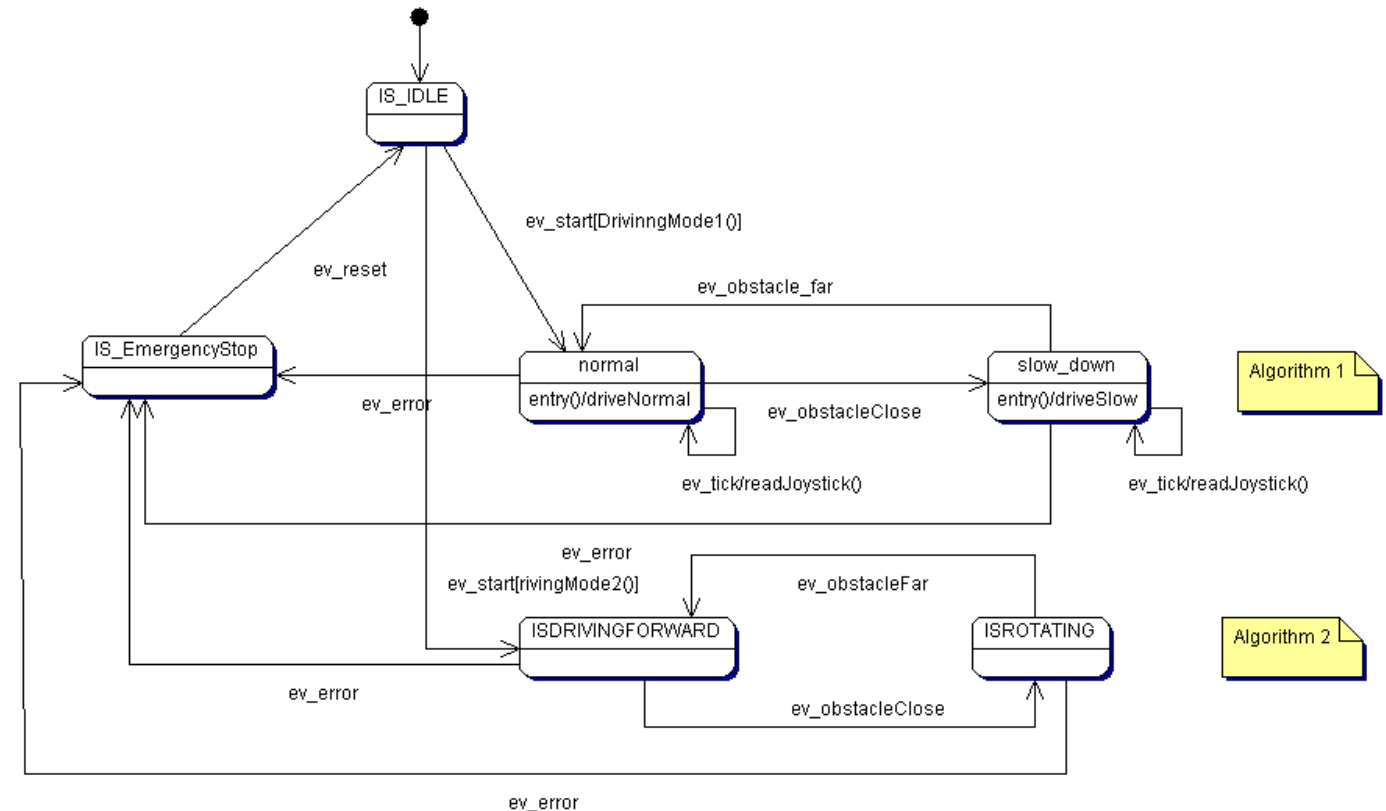
- The car is controlled through 3 main states
- Within the driving states, different algorithms containing own state machine can be executed



Translation of the Car State machine into a flat state machine

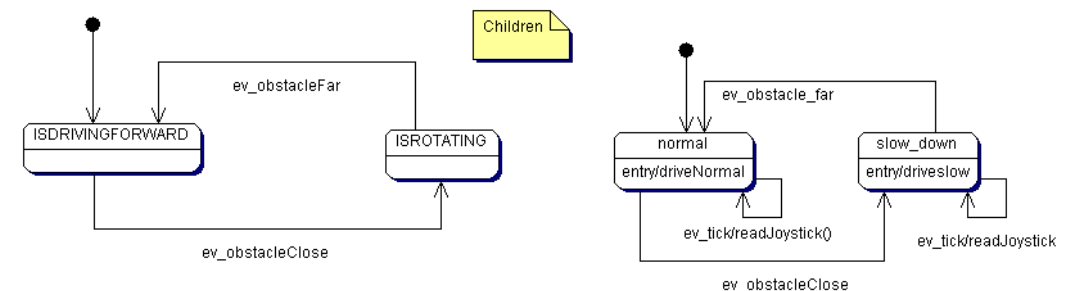
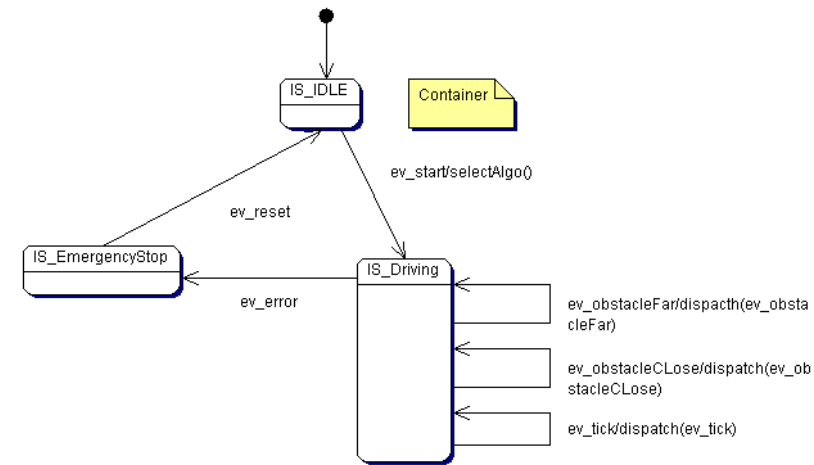
Traditional Approach

- Same behavior
- Can be implemented with the given coding patterns
- But readability is hard!



Translation into active objects

- We avoid the hierarchical design by introducing an event dispatcher as action
- The event dispatcher will send events which are not handled by the main state machine to the child active objects
- The individual state machines can easily be implemented
- AND we keep the design understandable



Let's dig a bit deeper

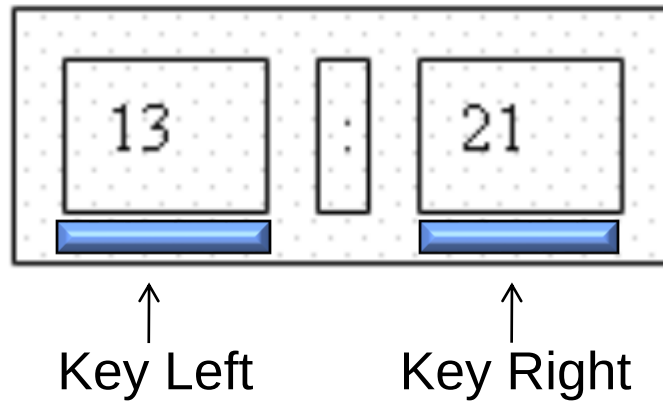
Example Digital Clock

Requirements

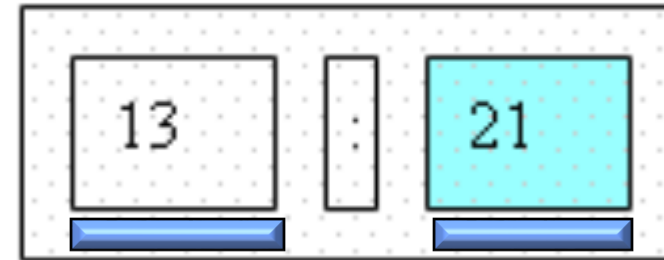
- The sample considers a digital clock.
- The digital clock shows the current time (two digit hours, a colon and two digits minutes).
- In display mode the current time is updated every minute.
- Also a manual edit mode (separately for minutes and hours) should be offered.
- Two keys are available for editing: KeyLeft and KeyRight.
 - KeyLeft: Cycle through the modes Display, ChangeHours, ChangeMinutes
 - KeyRight: Change Time

Digital Clock

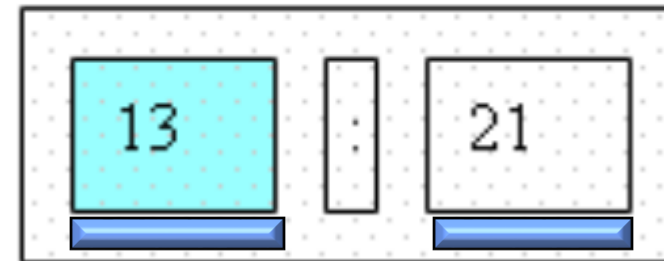
Time displaying mode



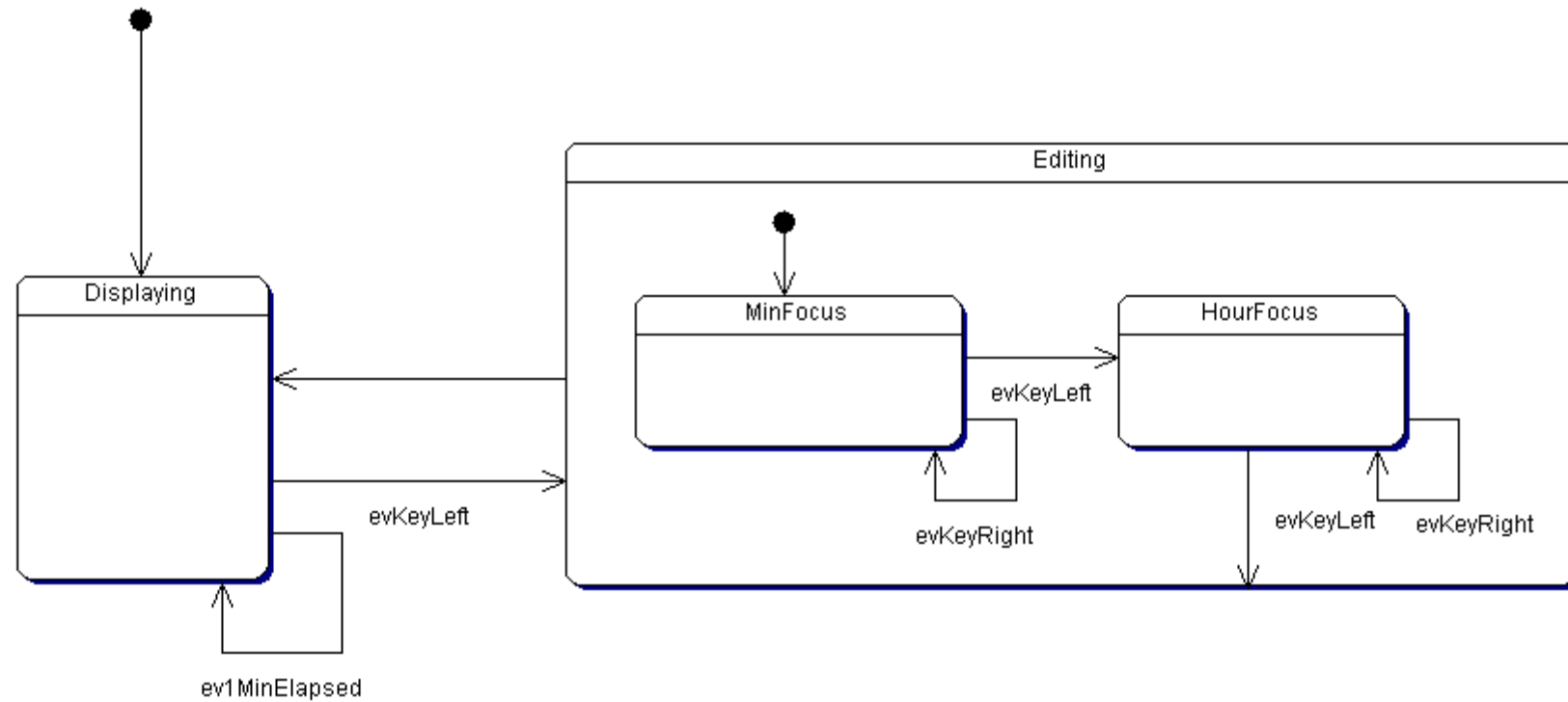
Minutes editing mode



Hours editing mode



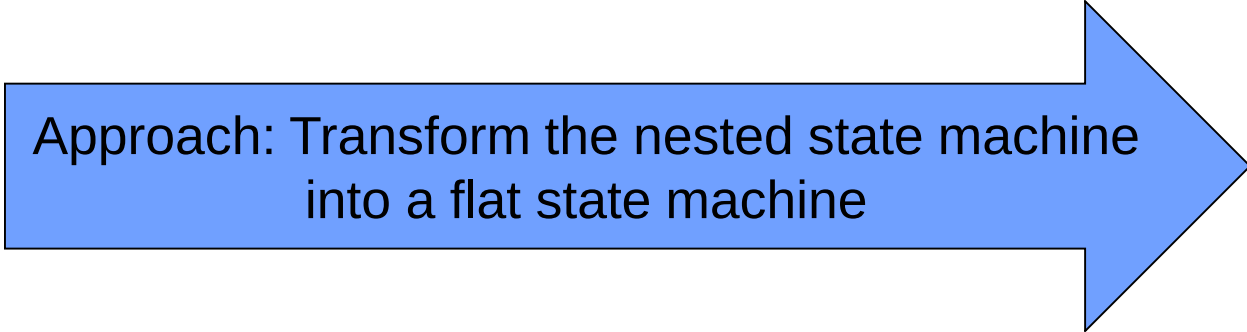
Hierarchical Design



Pitfalls of the hierarchical implementation

Complex states are such states, that contain (possibly nested) state machines. When complex states are present in a State Machine, then eventually more than one state is active at one moment of time. This causes a more complex implementation, which is harder to understand, harder to maintain.

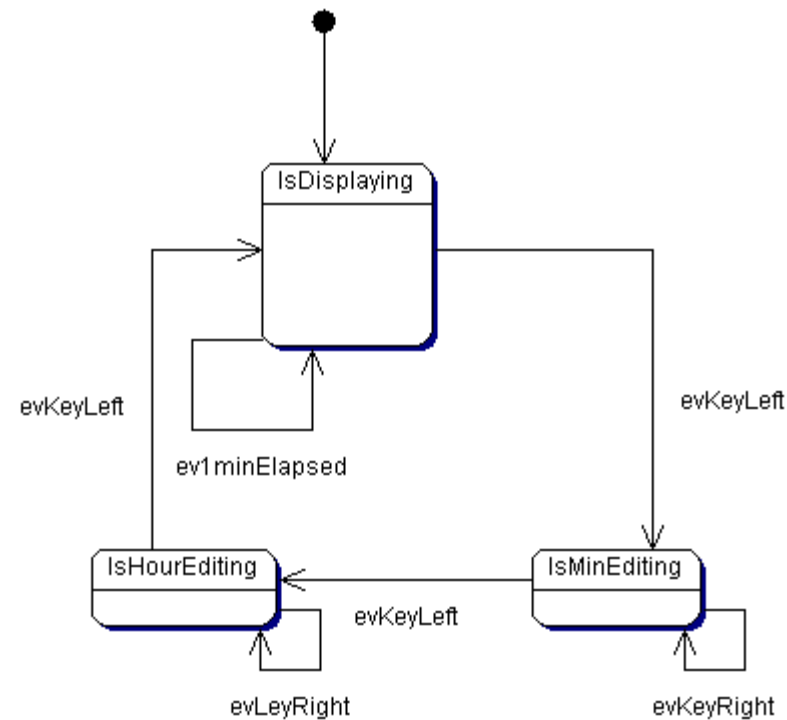
Clock Example: Due to the use of a complex state (the state "Editing" has sub-states) this design cannot be implemented with the known pattern.



Approach: Transform the nested state machine
into a flat state machine

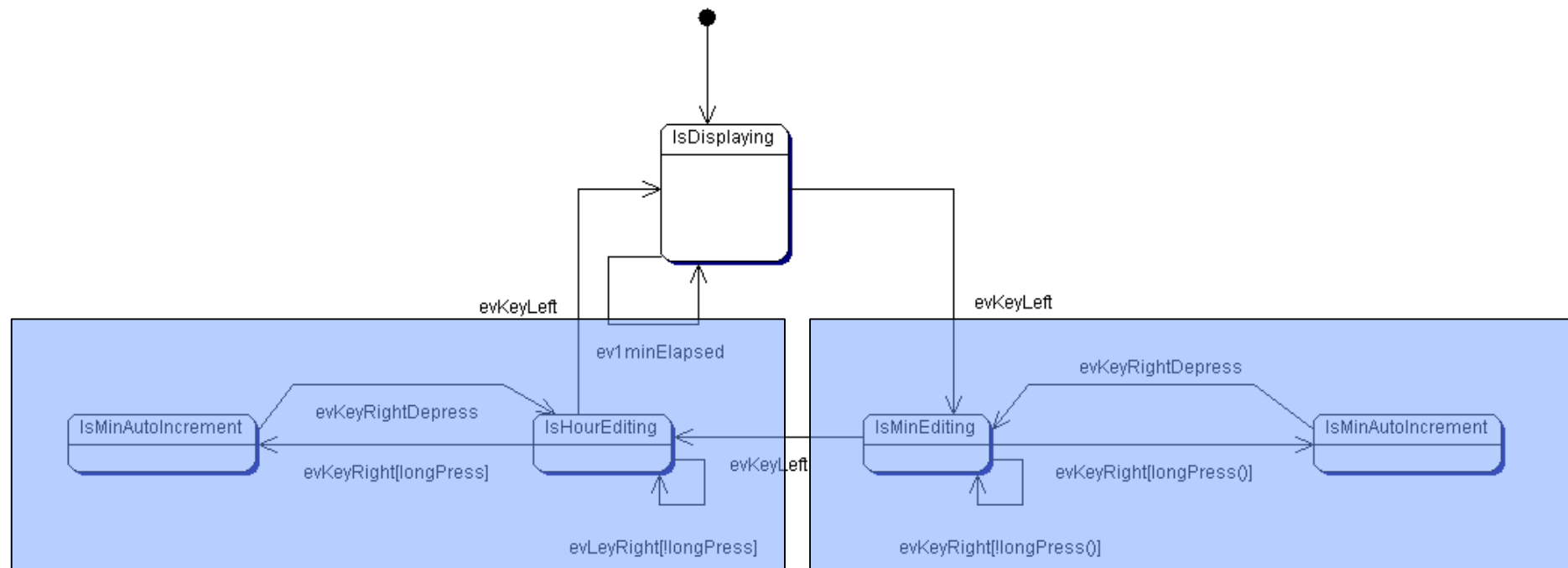
Trivial flat design

Due to the very simple internal state machine (editing or showing the time) of the minutes and hours widget, transforming this machine into a flat machine is pretty obvious.



Limitations of the simple transformation

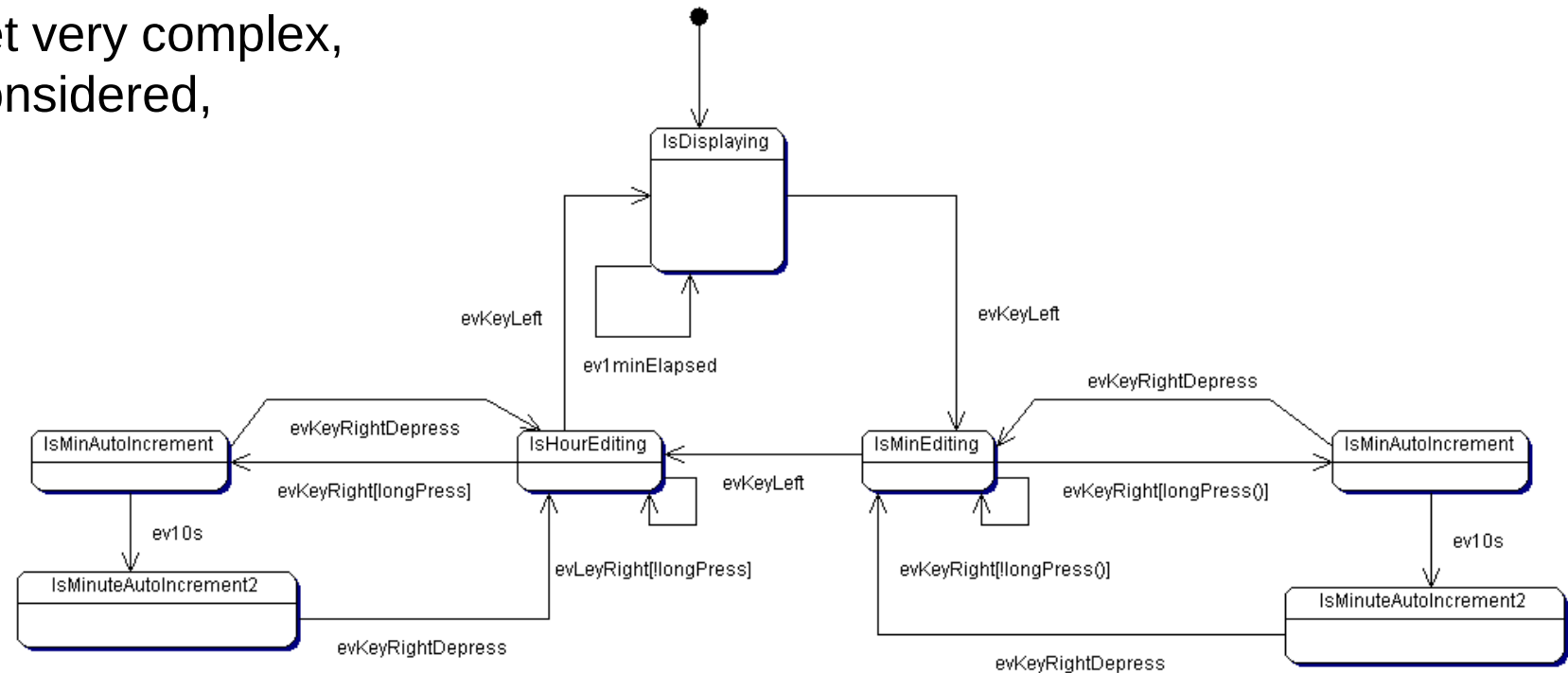
This simple transformation is possible, because editing the hours and minutes is represented by a single state only. Now, let's assume, that we have some additional logic, like: if we press the button longer, the setting will be incremented with increasing speed.



The story continues....

Result:

- The flat state machines get very complex,
- many states need to be considered,
- Copy and paste
- maintainability will suffer

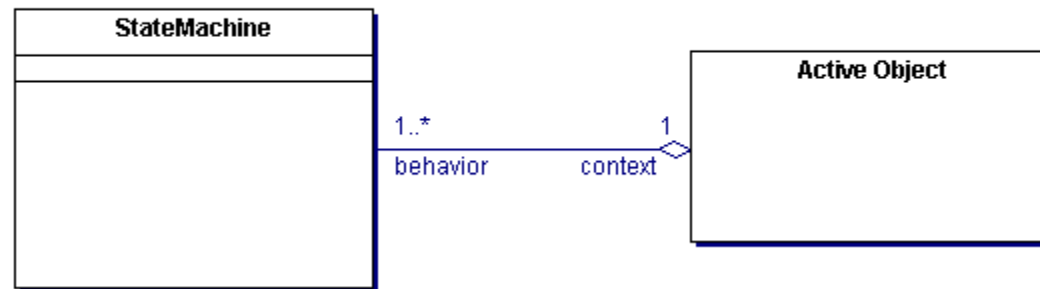


We need a solution for reusing parts of the state machine without the need for hierarchical states.

Active Object

Definition

In UML a State Machine is always attached to an object (or to all objects of a class). An object in conjunction with a State Machine attached to it, is commonly called an **active object** (e.g. the presented clock).



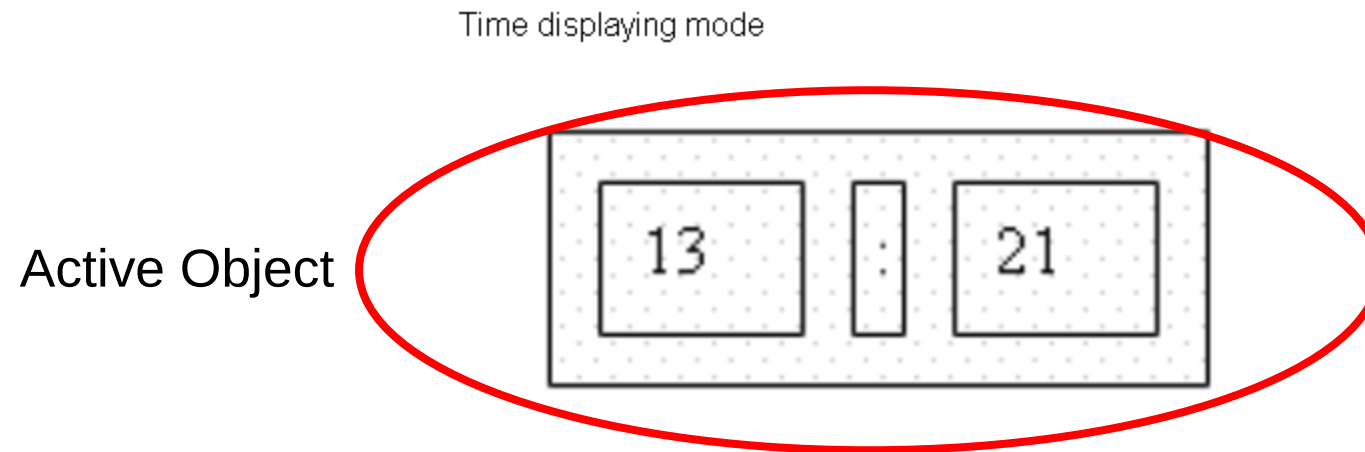
Active Object

- Objects with the same behavior, i.e. instances of one class, share the same State Machine.
- In the implementation the state should be kept in the private object attributes, and is therefore internal and not directly accessible from outside.
- The functionality of active objects is triggered by events. The action and/or state transition depends on the particular event and the current state of the object. The event processing, though logically the same for all active objects in a class, must take the object into account.
- State changes of active objects occur only as side-effect of events, which are processed by the state machine. No state change in any other way is allowed.

This is not really different from our state machine definition so far.
The only extension is – we now have several state machines
implemented in several active objects.

The clock as active object

The whole display content may be considered as one active object. This seems the simplest design at a first glance.



Divide and Conquer

Divide and Conquer is a powerful general principle of Computer Science:

- Break-up a complex problem step by step into smaller sub-problems, which may be treated separately.

Applied to design of state machines, it can be rephrased:

- Break-up a complex state machine associated with a single Active Object into a hierarchy of simpler active objects.

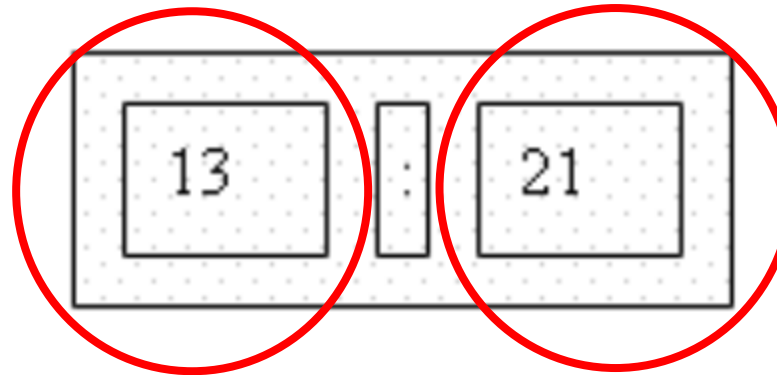
The aim should be to avoid complex state machines in a design. They should be replaced by several active objects with simple flat state machines.

The widgets as active objects

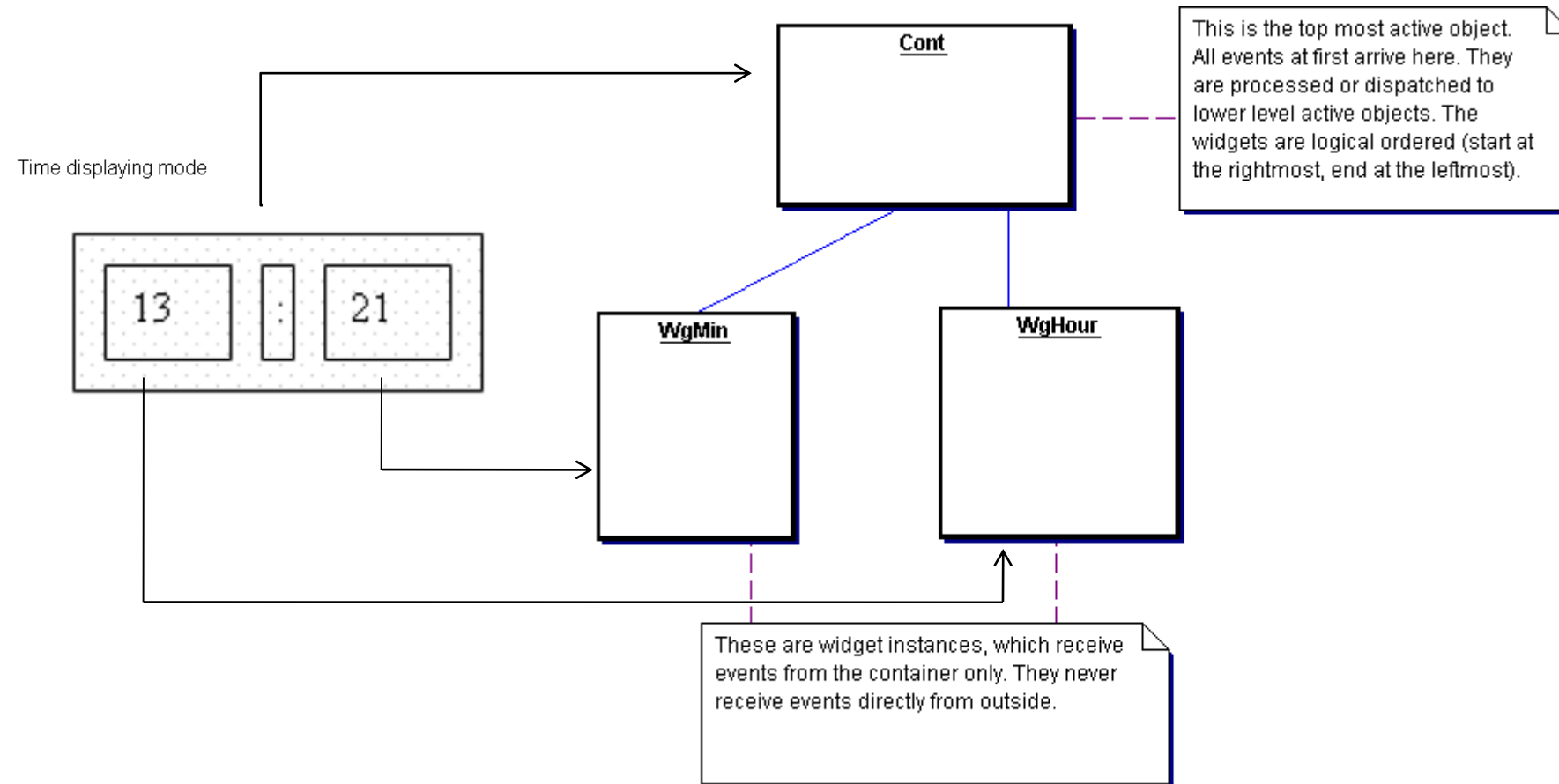
We now consider every widget as an active object...

Active Object

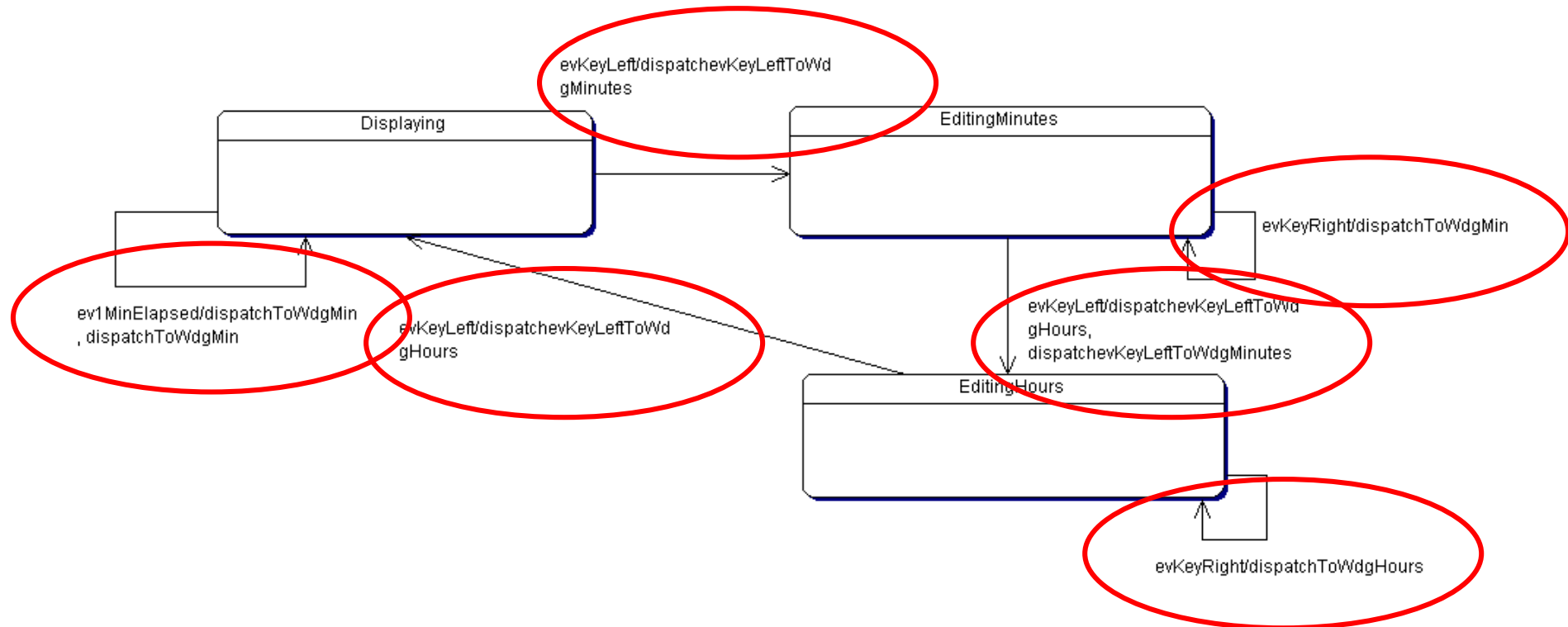
Time displaying mode



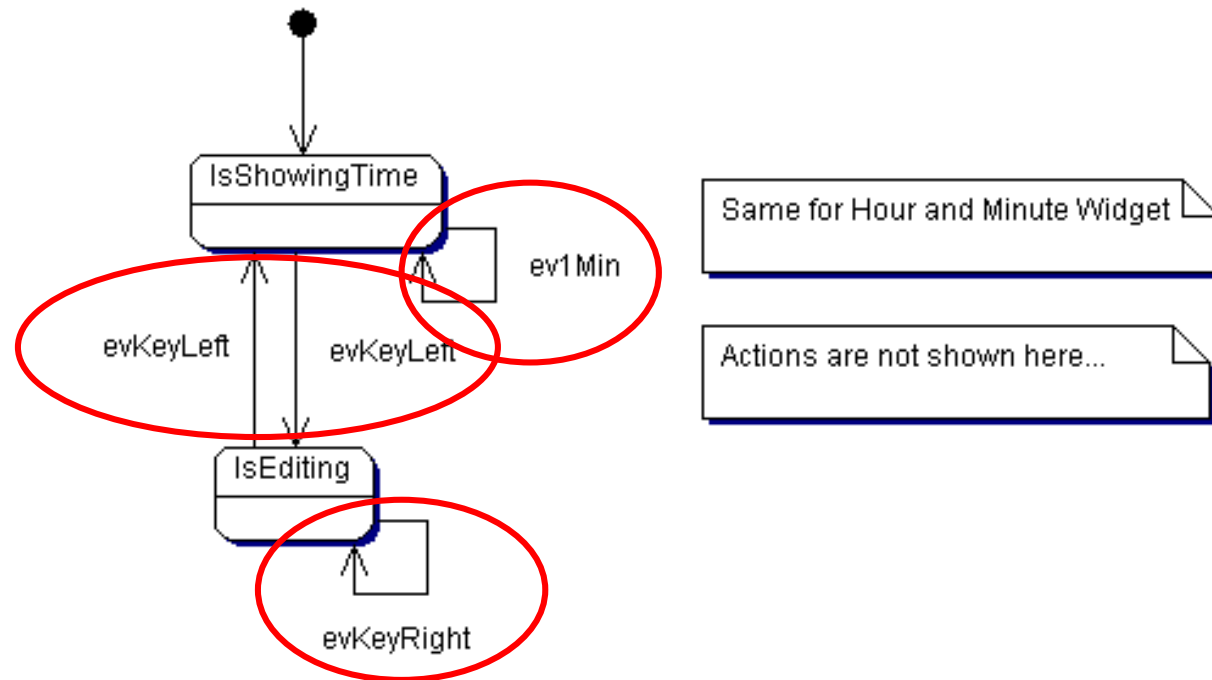
Divide and Conquer



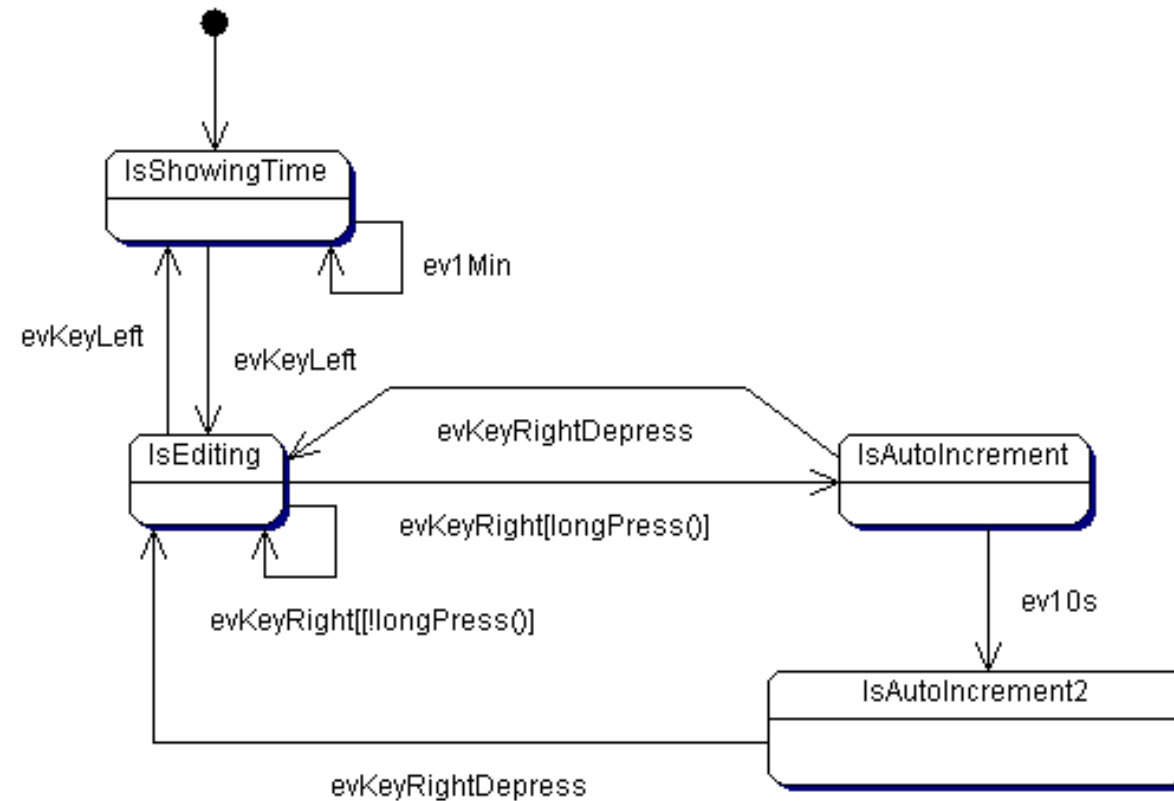
State Machine of the Container



State Machine of the Hours / Minute Widgets



Example of the Hour/Min Widget for the extended clock function



Extensions are added to the common widget state machine, i.e. they will be deployed for both the hour and minute widget at the same time. This will guarantee identical behavior for the hour and minute widget – what we definitely want to achieve!

Pro's and Con's of Hierarchical Objects

Pro's

- The designed state machines can be implemented using simple implementation patterns

Con's

- The design of the state machine becomes more complex
- Two state machines must be implemented
- A mechanisms to dispatch events must be made available
- Does not work for all hierarchical state machines

Some thought in between

Active object – sound like we need classes.

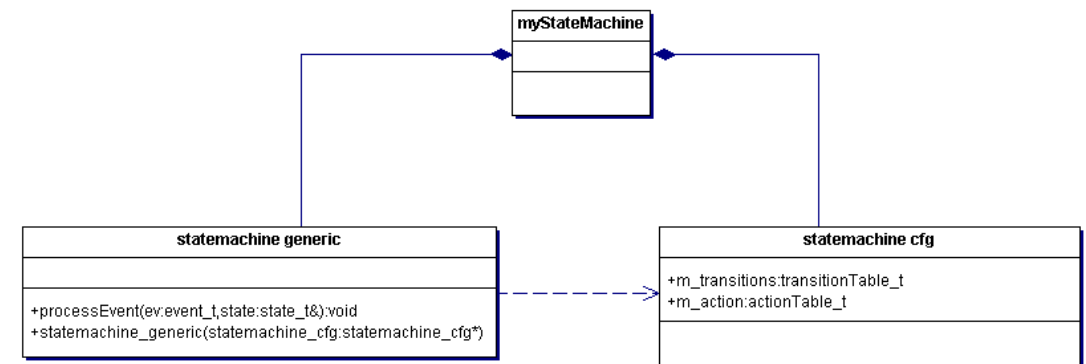
- Does C provide classes?
- What do we need for a class?
- Namespace
- Objects
- Scope
- Attributes / Methods
- Visibility in Together

Communication between Container and Widget

- How can we implement this?
- Can we provide 2 state machine configuration but use the same event processing code?
- How do we store the different states?

Active Object C-Class

- A structure for myStateMachine contains
 - My events
 - My states
 - My transition table (pointer)
 - My action tables (pointer)
- Objects are represented as instances of the struct, having different values (a bit tricky for the events and states, as enums are not considered as different values, but as different types!)
- The statemachine generic comes as a function taking all the above attributes as parameters



Active Object Tables – Clock Container

```
//===== Container =====

/** ===== CLOCK_C_ISDISPLAYING ===== */
const STATE_stateInnerTransition_t STATE_CLOCK_C_ISDISPLAYING_Transitions[] = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYLEFT,      CLOCK_C_ISEDITINGHOURS, 0,          CLOCK__dispatch_KL_Hours    },
  { EV_1MIN,         CLOCK_C_ISDISPLAYING, 0,          CLOCK__increment1Minute    },
};

/** ===== CLOCK_C_ISEDITINGHOURS ===== */
const STATE_stateInnerTransitionTable_t STATE_CLOCK_C_ISEDITINGHOURS_Transitions = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYLEFT,      CLOCK_C_ISEDITINGMINUTES, 0,          CLOCK__dispatch_KL_HoursMin },
  { EV_KEYRIGHT,     CLOCK_C_ISEDITINGHOURS, 0,          CLOCK__dispatch_KR_Hours    },
  { EV_KEYRIGHTLONGPRESS, CLOCK_C_ISEDITINGHOURS, 0,          CLOCK__dispatch_KRLP_Hours  },
  { EV_250MS,        CLOCK_C_ISEDITINGHOURS, 0,          CLOCK__dispatch_250MS_Hours },
};

/** ===== CLOCK_C_ISEDITINGMINUTES ===== */
const STATE_stateInnerTransitionTable_t STATE_CLOCK_C_ISEDITINGMINUTES_Transitions = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYLEFT,      CLOCK_C_ISDISPLAYING, 0,          CLOCK__dispatch_KL_Min     },
  { EV_KEYRIGHT,     CLOCK_C_ISEDITINGMINUTES, 0,          CLOCK__dispatch_KR_Min     },
  { EV_KEYRIGHTLONGPRESS, CLOCK_C_ISEDITINGMINUTES, 0,          CLOCK__dispatch_KRLP_Min   },
  { EV_250MS,        CLOCK_C_ISEDITINGMINUTES, 0,          CLOCK__dispatch_250MS_Min  },
};

const STATE_stateOuterTransitionTable_t CLOCK_Container_Transitions = {
/*   fromState       Pointer to table       Size of table [Elements] */
  { CLOCK_C_ISDISPLAYING, &STATE_CLOCK_C_ISDISPLAYING_Transitions, sizeof(STATE_CLOCK_C_ISDISPLAYING_Transitions)/sizeof(STATE_stateInnerTransition_t) },
  { CLOCK_C_ISEDITINGHOURS, &STATE_CLOCK_C_ISEDITINGHOURS_Transitions, sizeof(STATE_CLOCK_C_ISEDITINGHOURS_Transitions)/sizeof(STATE_stateInnerTransition_t) },
  { CLOCK_C_ISEDITINGMINUTES, &STATE_CLOCK_C_ISEDITINGMINUTES_Transitions, sizeof(STATE_CLOCK_C_ISEDITINGMINUTES_Transitions)/sizeof(STATE_stateInnerTransition_t) },
};

const uint16_t CLOCK_Container_Transitions_Size = sizeof(CLOCK_Container_Transitions)/sizeof(STATE_stateOuterTransition_t);
```

Active Object Tables – Clock Widget(s) / Identical for hour and minute

```
//===== WIDGET =====

/** ===== CLOCK_W_ISDISPLAYING ===== */
const STATE_stateInnerTransitionTable_t STATE_CLOCK_W_ISDISPLAYING_Transitions = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYLEFT,      CLOCK_W_ISEDITING,  0,             CLOCK__setEditFonts  },
};

/** ===== CLOCK_W_ISEDITING ===== */
const STATE_stateInnerTransitionTable_t STATE_CLOCK_W_ISEDITING_Transitions = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYRIGHT,     CLOCK_W_ISEDITING,  0,             CLOCK__incrementWidget },
  { EV_KEYRIGHTLONGPRESS, CLOCK_W_ISAUTOINCREMENT, 0,             0 },
  { EV_KEYLEFT,      CLOCK_W_ISDISPLAYING, 0,             CLOCK__setDisplayFonts },
};

/** ===== CLOCK_W_ISAUTOINCREMENT ===== */
const STATE_stateInnerTransitionTable_t STATE_CLOCK_W_ISAUTOINCREMENT_Transitions = {
/*   Event           ToState           Guard           Action           */
  { EV_KEYRIGHT,     CLOCK_W_ISEDITING,  0,             CLOCK__incrementWidget },
  { EV_250MS,        CLOCK_W_ISAUTOINCREMENT, 0,             CLOCK__incrementWidget },
};

const STATE_stateOuterTransitionTable_t CLOCK_Widget_Transitions = {
/*   fromState       Pointer to table       Size of table [Elements] */
  { CLOCK_W_ISDISPLAYING, &STATE_CLOCK_W_ISDISPLAYING_Transitions, sizeof(STATE_CLOCK_W_ISDISPLAYING_Transitions)/sizeof(STATE_stateInnerTransition_t) },
  { CLOCK_W_ISEDITING,    &STATE_CLOCK_W_ISEDITING_Transitions,    sizeof(STATE_CLOCK_W_ISEDITING_Transitions)/sizeof(STATE_stateInnerTransition_t) },
  { CLOCK_W_ISAUTOINCREMENT, &STATE_CLOCK_W_ISAUTOINCREMENT_Transitions, sizeof(STATE_CLOCK_W_ISAUTOINCREMENT_Transitions)/sizeof(STATE_stateInnerTransition_t) },
};
const uint16_t CLOCK_Widget_Transitions_Size = sizeof(CLOCK_Widget_Transitions)/sizeof(STATE_stateOuterTransition_t);
```

Dispatcher mechanisms

- Dispatching now simply means – depending on the active object we want to dispatch to, we have to call the generic state machine function with different tables and objects (mainly storing the state)

```
**  
** Dispatch event to widget  
**/  
static void CLOCK__dispatch_KL_HoursMin()  
  
    LOG_I("CLOCK__dispatch_KL_HoursMin", "");  
  
    CLOCK__dispatch_wrapper(EV_KEYLEFT, CLOCK_HOURWIDGET);  
    CLOCK__dispatch_wrapper(EV_KEYLEFT, CLOCK_MINWIDGET);
```

```
/**  
** General Dispatcher, saving the states  
**/  
static void CLOCK__dispatch_wrapper(STATE_event_t ev, CLOCK_active_widget_t widget)  
{  
  
    //Read the state object (container)  
    CLOCK_data_t clock = RTE_CLOCK_get(&SO_CLOCK_signal);  
  
    //Set the active widget to the hour widget  
    clock.m_activeWidget = widget;  
  
    //Write the changed widget back to the clock object  
    RTE_CLOCK_set(&SO_CLOCK_signal, clock);  
  
    //And fire it to the container state machine  
    //The widget state machines will be called by the container actions  
    //We need a local copy of the widget state as the widget object is also changed by the widget actions  
  
    if (CLOCK_HOURWIDGET == widget)  
    {  
        CLOCK_state_t localState = clock.m_hours.m_state;  
        CLOCK_processEvent(&CLOCK_Widget_Transitions, CLOCK_Widget_Transitions_Size, 0, ev, &localState);  
  
        //Re-Read the state object in order to get the changes by the child  
        clock = RTE_CLOCK_get(&SO_CLOCK_signal);  
        clock.m_hours.m_state = localState;  
  
        //Write the changed state back to the clock object  
        RTE_CLOCK_set(&SO_CLOCK_signal, clock);  
    }  
    else if (CLOCK_MINWIDGET == widget)  
    {  
        CLOCK_state_t localState = clock.m_minutes.m_state;  
        CLOCK_processEvent(&CLOCK_Widget_Transitions, CLOCK_Widget_Transitions_Size, 0, ev, &localState);  
  
        //Re-Read the state object in order to get the changes by the child  
        clock = RTE_CLOCK_get(&SO_CLOCK_signal);  
        clock.m_minutes.m_state = localState;  
  
        //Write the changed state back to the clock object  
        RTE_CLOCK_set(&SO_CLOCK_signal, clock);  
    }  
}
```

Active Objects and RTE

- Until now, we have focused on Data Signals, transmitting data from one runnable to another.
- But at the end of the day, a signal was nothing but some global data with a specific API.
- To a certain extent, we can use this concept also for statemachines
 - The state of a specific state machine is nothing but a global variable (aka signal)
 - The access to this state variable should be restricted to a single runnable (the state machine runnable)
 - Events, which are fired to this state machine are represented by “normal” data signals



Event Signal

Advantage over OS events

- We need only one OS event to signal a new event: `ev_EVENT_onUpdate`
- We abstract OS events from our API

Disadvantage

- These events can be overwritten
- Minor runtime and memory penalty

Extension

- Let's use a queue (e.g. ringbuffer) to store events

```
typedef enum {  
    EV_NONE,  
    EV_KEYLEFT,  
    EV_KEYRIGHT,  
    EV_KEYRIGHTLONGPRESS,  
    EV_250MS,  
    EV_1MIN  
} EVENT_enum_t;
```

```
typedef struct  
{  
    EVENT_enum_t m_ev;  
} EVENT_data_t;
```

```
#define EVENT_INIT_DATA ((EVENT_data_t){EV_NONE})
```

Challenge 1 – Limitation to get and set

Let's assume the following scenario:

- We want to create a buffered signal, e.g. a event FIFO queue
- A FIFO can be implemented as a ringbuffer
- We can easily place a ringbuffer into a signal, but accessing it becomes rather hacky.

```
//Get signal
ringbuffer_data_t ring = RTE_RINGBUFFER_get(&so_event_fifo)

//Proccess ringbuffer, get a single element (which will change read index as well)
uint8_t data = ringbuffer_get(&ring);

//Store signal
RTE_RINGBUFFER_set(&so_event_fifo, ring)
```

Hacky and possible race: what happens if another event is written while we are processing the data?

Idea – Let's use C++ objects and add an API call to get a reference to the object

- C++ objects come with class specific methods, i.e. we can extend the signal API per signal class
- Furthermore we can make certain properties of the signal private
- Many advantages!!!! But we need an embedded C++ compiler.

```
//Get signal reference
```

```
ringbuffer_ref_data_t ring = so_event_fifo.getReference();
```

```
//Process ringbuffer, get a single element (which will change read  
//index as well). Performed on global signal (reference). This  
//operation may be protected using mutexes
```

```
uint8_t data = ring.get();
```

```
//"Return" reference - usually not required, as we performed operations  
//on the global object directly. May be used, e.g. to trigger RTE event  
//so_event_fifo.returnReference(ring);
```

Another solution – Let's store a pointer to a ringbuffer as signal

- This would reduce the risk for a race, as the application would only read the ringbuffer address and then work on the ringbuffer object directly
- Writing the ringbuffer object after adding data to it would even allow us to use the RTE notification mechanism
- However, MPU protection of the “real” signal does not really work anymore – unless we manually link it into the correct section.

```
//Get ringbuffer address, ringbuffer_data_t is a pointer  
ringbuffer_data_t pRing = RTE_RINGBUFFER_get(&so_event_fifo)
```

```
//Process ringbuffer, get a single element (which will change read  
//index as well). We modify the global object directly  
uint8_t data = ringbuffer_get(pRing);
```

```
//Only required for sending RTE trigger  
//RTE_RINGBUFFER_set(&so_event_fifo, ring)
```

Challenge 2 – Storing State and Payload in the same object

```
typedef struct
{
    CLOCK_state_t
    STATE_stateOuterTransitionTable_t*
    STATE_stateOuterAction_t*
    uint8_t
    CLOCK_font_t
} CLOCK_widget_t;

typedef struct
{
    CLOCK_state_t
    STATE_stateOuterTransitionTable_t*
    STATE_stateOuterAction_t*
    CLOCK_active_widget_t
    CLOCK_widget_t
    CLOCK_widget_t
} CLOCK_data_t;
```

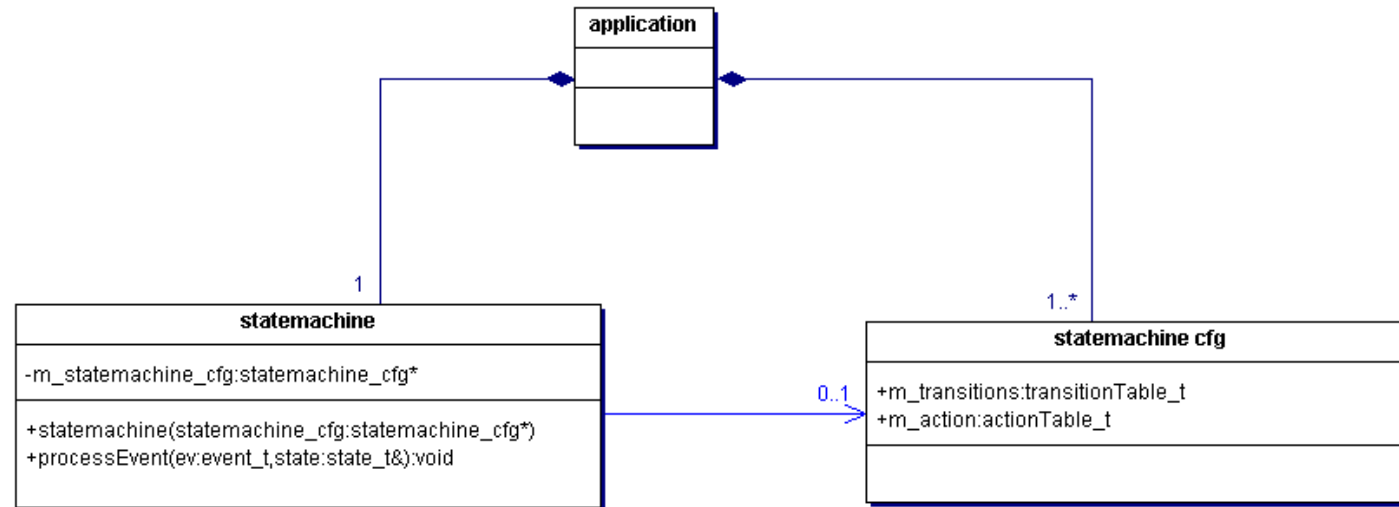
Diagram illustrating the relationship between the two structs:

- The `CLOCK_data_t` struct contains a `CLOCK_widget_t` member.
- The `CLOCK_widget_t` struct contains state-related members: `CLOCK_state_t`, `STATE_stateOuterTransitionTable_t*`, `STATE_stateOuterAction_t*`, `uint8_t`, and `CLOCK_font_t`.
- The `CLOCK_data_t` struct contains payload-related members: `CLOCK_active_widget_t`, `CLOCK_widget_t`, and `CLOCK_widget_t`.
- The `CLOCK_data_t` struct also contains a `CLOCK_widget_t` member, which is represented by a diamond symbol (composition) pointing to the `CLOCK_widget_t` struct.

All data belonging to a clock are stored in the same object (good cohesion)

The object data will be modified by the state actions. The state will be modified by the state machine. Possible race. Getter and setters must be used with care.

Challenge 3 – Type Definitions with multiple instances of statemachines



Problem:

`event_t` and `state_t` are defined in `statemachine_cfg.h` and will differ per `statemachine`.

(Hacky) Solution 1:

Instead of using enum types as parameters, take `uint16_t` and cast the enums.

(Hacky) Solution 2:

Define a global `state_t` and `event_t` containing ALL states and ALL events for ALL statemachines

Problem:

`transitionTable_t` and `actionTable_t` are defined in `statemachine.h`. The file cannot be included, otherwise we would get a circular dependency.

Solution:

Add an extra header file containing central type definitions, which is included by both classes.

Challenge 4 – The Update of the TFT display

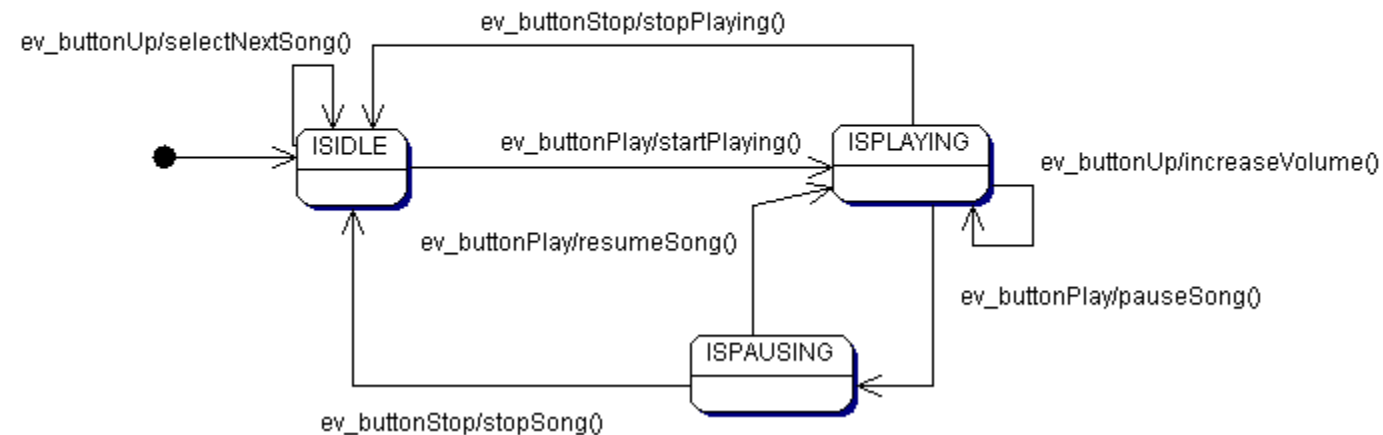
- ...takes between 20 and 60ms
- In this a problem? Why?
- Solution?



State Machine Testing

No matter which approach we use for implementing a state machine, they are great to create systematic testcases.

- Every transition is a testcase
- Every “empty” transition also is a testcase (as nothing may happen!)



State Machine Testing - Transitions

Event / State	ISIDLE	ISPLAYING	ISPAUSING
ev_buttonUp	• ISIDLE • selectNextSong()	• ISPLAYING • increaseVolume()	No action
ev_buttonPlay	• ISPLAYING • startPlaying()	• ISPAUSING • pauseSong()	• ISPLAYING • resumeSong()
ev_buttonStop	No action	• ISIDLE • stopSong	• ISIDLE • stopSong

Testcase Input:

- fromState
- Event
- Guard (not shown here)

Testcase Expected Result:

- toState
- transition action
- state action (not shown)
- No action

Guards create extra testcases

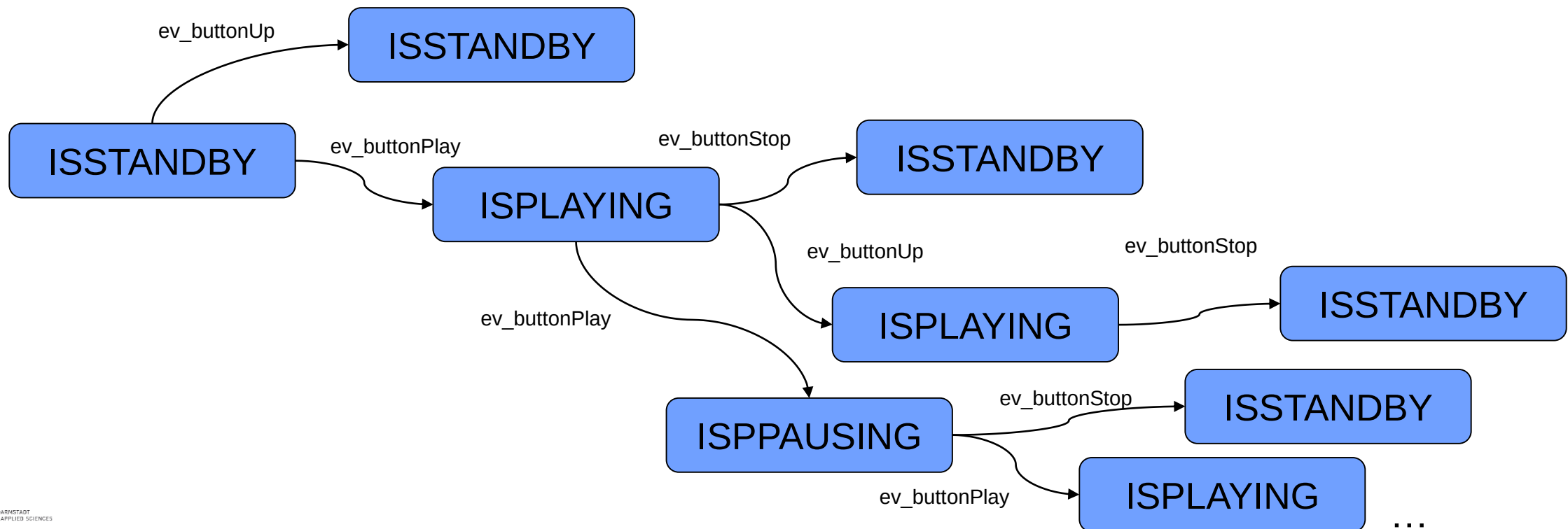


Design flaw!

Event / State	ISIDLE	ISPLAYING	ISPAUSING
ev_buttonUp	- ISIDLE - selectNextSong()	- ISPLAYING - increaseVolume()	No action
ev_buttonPlay songAvailable()	- ISPLAYING - startPlaying()	- ISPAUSING - pauseSong()	- ISPLAYING - resumeSong()
ev_buttonPlay !songAvailable()	No action	???	???
ev_buttonStop	No action	- ISIDLE - stopSong	- ISIDLE - stopSong

State Machine Testing – Transition Tree

- Although a systematic test of all transitions already is a pretty good coverage, it still might be that the combination of certain transitions causes a problem.
- Therefore, in a second step, a transition tree is generated, which covers all transition combinations until the original state is reached again



Resulting Testcases

Id	Event Sequence	Expected Result
1	• ev_buttonUp	• selectNextSong()
2	• ev_buttonPlay • ev_buttonStop	• playSong() • stopSong()
3	• ev_buttonPlay • ev_buttonUp • ev_buttonStop	• playSong() • increaseVolume() • stopSong()
...		

Note: Starting and End state are omitted, because in all cases this will be ISSTANDBY

Note 2: In case of guards, every event/guard combination must be considered separatly

Wrapup

- From “static” state machines to active objects
- State / Transition tables as data structures representing the logic of an active object
- Concept of event dispatching
- Active objects / RTE and signals
- State machine and Events as signals
- Challenges mainly related to limited get/set API
- State machine testing
- Systematic transition testcases
- Transition tree test concept